MGL 7230: Tests Logiciels

Chapitre #1

Introduction aux tests logiciels

Objectives

1. Comprendre l'objectif et les buts des tests logiciels, y compris leur rôle dans l'assurance qualité et l'identification des défauts (bogues).

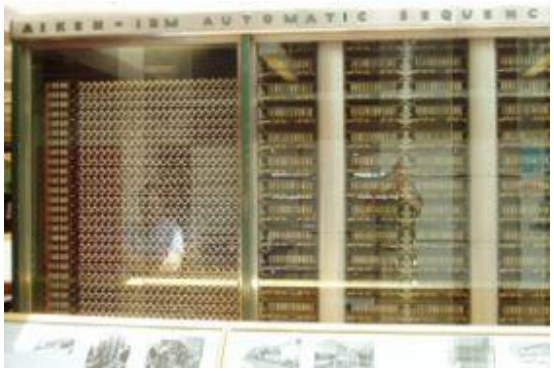
2. Explorer les différents types, niveaux et activités des tests logiciels dans le cadre du cycle de vie du développement logiciel (SDLC).

3. Acquérir des connaissances sur les concepts clés tels que les cas de test, l'automatisation des tests et les outils utilisés pour des tests logiciels efficaces.

Plan

- Qu'est-ce qu'un bogue ?
- Exemples de bogues réelles
- Impact des logiciels défectueux
- Objectifs des tests
- Définition des tests
- Coût des tests
- Niveaux de tests basés sur la maturité du processus de test
- Tests pour l'amélioration/l'assurance qualité
- Activités de test
- Relation des tests avec d'autres activités
- Tests dans le SDLC
- Types de tests logiciels
- Défaut vs. Erreur vs. Échec
- Vérification et Validation
- Qu'est-ce qu'un cas de test ?
- Oracle de test
- Testabilité des logiciels
- Exécution et automatisation des tests
- Équipe de test
- Outils de test

Qu'est-ce qu'un bogue informatique ?



En 1988, le journal de bord, avec l'insecte toujours **scotché** à l'entrée, était exposé au Naval Surface Warfare Center Computer Museum à Dahlgren, Virginie.

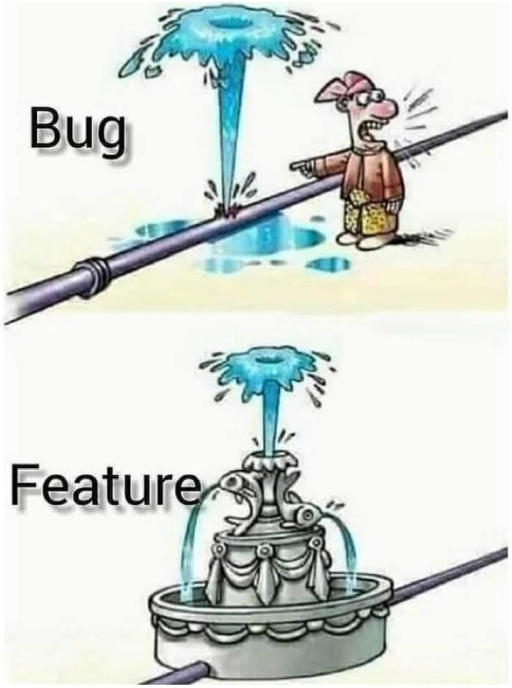
- En 1947, l'Université Harvard utilisait un ordinateur de la taille d'une pièce, appelé **Mark II**.
  - Relais mécaniques
  - Tubes à vide
  - Les techniciens programmaient l'ordinateur en le reconfigurant
- Un insecte (mite) est entré dans l'ordinateur et a été électrocuté par la haute tension.
- Ainsi est né le premier **bogue informatique** !

http://www.history.navy.mil/photos/images/h96000/h96566kc.htm

Les bogues …

- Bogue (utilisé de manière informelle)
- Défaut
- Faute
- Problème
- Erreur
- Incident
- Anomalie
- Échec
- Incohérence

Feature 😇



Échec spectaculaire d'un logiciel : Ariane 5

- Le 4 juin 1996, le premier vol de la fusée Ariane 5 de l'Agence Spatiale Européenne s'est terminé en catastrophe quelques secondes seulement après le décollage.

https://www.youtube.com/watch?v=gp_D8r-2hwk

Désastre de la fusée Ariane 5

- Causé par **une erreur de dépassement numérique (overflow)**
  - Tentative (lors du décollage) de convertir la vitesse latérale de la fusée d'un format 64 bits à un format 16 bits.
  - Le nombre était trop grand pour être stocké, ce qui a entraîné une erreur de dépassement.
  - L'erreur a été mal interprétée par l'ordinateur de bord de la fusée comme un signal pour changer la trajectoire de la fusée
- **Coût**
  - 100 millions de dollars de perte de mission
  - Retard de plusieurs années pour le programme
- Échec du lancement d'Ariane expliqué par Ian Sommerville
  - https://www.youtube.com/watch?v=W3YJeoYgozw

Bogues récents :
Pannes des pompes à carburant en Nouvelle-Zélande (2024)

- **Bogue:** Le 29 février 2024, un bogue **lié à l'année bissextile** dans le logiciel de **traitement des paiements** a rendu les pompes des stations-service en libre-service à travers la Nouvelle-Zélande incapables d'accepter les paiements par carte, entraînant d'importantes perturbations.

- **Impact:** La panne a duré plus de 10 heures, forçant certaines stations-service à fermer complètement. Les fournisseurs de carburant affectés comprenaient Allied Petroleum, BP, Gull, Waitomo et Z Energy.

https://www.wired.com/story/leap-year-glitch-gas-pumps-broken-new-zealand/#:~:text=But%20that%20unique%20calendar%20date,gas%20pumps'%20payment%20processing%20software.

Bogues récents : Panne de CrowdStrike Falcon (2024)

- En juillet 2024, **une mise à jour défectueuse** du produit **CrowdStrike Falcon** sur Windows a provoqué des écrans bleus (BSOD) et des boucles de redémarrage.

- L'incident n'était pas un piratage : c'était **un bug de mise à jour** déployée largement.

- **Impact mondial** : fortes perturbations (aéroports/compagnies aériennes, banques, hôpitaux, médias, grandes entreprises).

- **Correction** : CrowdStrike a retiré/poussé un correctif, mais beaucoup d'organisations ont dû faire une remédiation manuelle machine par machine.

https://www.theatlantic.com/ideas/archive/2024/07/microsoft-outage-technological-systems-fail/679110/?utm

Bogues récents :
Panne logicielle des trains Renfe Avril (Espagne, 2025)

- Le 1er janvier 2025, une panne a rendu toute la flotte de trains à grande vitesse Talgo Avril de Renfe inopérante, affectant plus de 14 000 passagers sur les principales lignes en Espagne.

- La perturbation était due à une défaillance de communication entre le système de contrôle et les chargeurs de batterie des trains, **liée au changement d'année.**

https://www.railtech.com/all/2025/01/02/new-years-breakdown-renfes-talgo-avril-trains-halted-by-battery-failure/?gdpr=accept

Exemples de vulnérabilités de sécurité

- **Vulnérabilité Heartbleed (2014) (https://heartbleed.com)**
  - **Bogue** : Une faille dans la bibliothèque OpenSSL permettait aux attaquants de lire la mémoire des systèmes utilisant des versions vulnérables, exposant ainsi des données sensibles telles que des mots de passe et des clés de chiffrement.
  - **Impact** : A affecté des millions de sites Web, entraînant des violations de sécurité à grande échelle.
- **Violation de données Equifax (2017) (https://www.blackduck.com/blog/equifax-apache-struts-vulnerability-cve-2017-5638.html)**
  - **Bogue** : Une vulnérabilité connue dans Apache Struts (CVE-2017-5638) est restée non corrigée, permettant aux pirates de l'exploiter et d'accéder à des données sensibles.
  - **Impact** : Les informations personnelles de 147 millions d'individus ont été compromises, entraînant des pertes financières et des dommages à la réputation.
- **Vulnérabilité Apache Log4Shell (2021) (https://www.ibm.com/think/topics/log4j)**
  - **Bogue** : Une faille dans la bibliothèque java de journalisation Log4j permettait l'exécution de code à distance via une chaîne de journalisation spécialement conçue.
  - **Impact** : Des millions d'applications et de serveurs ont été exposés aux attaques.

Effets néfastes des logiciels défectueux

- **Communications :** Perte ou corruption des supports de communication, non-livraison des données.
- **Applications spatiales :** Pertes humaines, retards de lancement.
- **Défense et guerre :** Mauvaise identification des amis et ennemis.
- **Transport :** Décès, retards, accélération soudaine, incapacité à freiner.
- **Énergie électrique :** Décès, blessures, pannes d'électricité, risques sanitaires à long terme (radiation).

Effets néfastes des logiciels défectueux (suite)

- **Gestion financière**: Fraude, violation de la vie privée, fermeture des bourses et des banques, taux d'intérêt négatifs.
- **Contrôle des élections**: Résultats erronés (intentionnels ou non intentionnels ☺).
- **Contrôle des prisons:** Tentatives et réussites d'évasion facilitées par la technologie, libération accidentelle de détenus, défaillances des verrous contrôlés par logiciel.
- **Application de la loi** : Arrestations et emprisonnements injustifiés.

Échecs coûteux des logiciels

- **NIST (2002):** estimation de 59,5 Md$/an pour l'économie US, avec 22,2 Md$ d'économies potentielles via des améliorations de l'infrastructure de test.
- **CISQ** – Cost of Poor Software Quality (2020): estime le coût total de la mauvaise qualité logicielle aux États-Unis à ~2,08 trillions $.
- **CISQ** – rapport 2022 : mise à jour à ≥ 2,41 trillions $ et dette technique accumulée ~1,52 trillion $.

NIS: Institut National des Standards et de la Technologie (États-Unis)
CISQ: Consortium for Information & Software Quality

Qu'est-ce que cela signifie ?

Les tests logiciels deviennent de plus en plus importants

Spécification (« si vous ne pouvez pas le dire, vous ne pouvez pas le faire »)

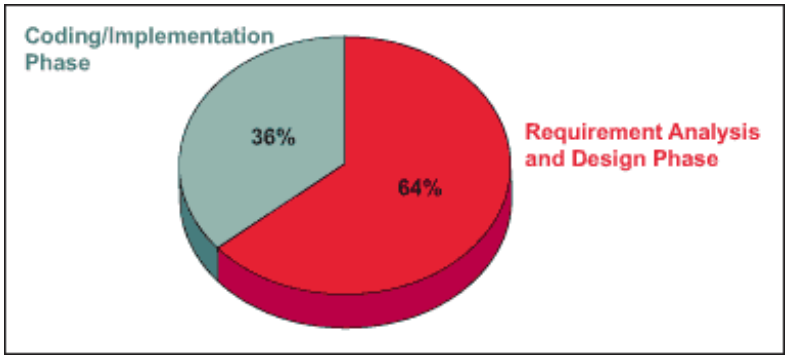
- Vous devez savoir ce qu'est votre produit avant de pouvoir dire s'il contient un bogue.
- Une spécification définit le produit en cours de création et comprend :
  - **Exigences fonctionnelles** qui décrivent les fonctionnalités que le produit prendra en charge, par ex. dans un traitement de texte :
    - Enregistrer, imprimer, vérifier l'orthographe, changer la police, …
  - **Exigences non fonctionnelles** qui sont des contraintes du produit, par ex.
    - Sécurité, fiabilité, convivialité, plateforme, …

Conditions qui causent des bogues logiciels

- Un bogue logiciel se produit lorsque au moins une de ces conditions est vraie :
  - Le logiciel ne fait pas quelque chose que la spécification dit qu'il doit faire.
  - Le logiciel fait quelque chose que la spécification dit qu'il ne doit pas faire.
  - Le logiciel fait quelque chose que la spécification ne mentionne pas.
  - Le logiciel ne fait pas quelque chose que la spécification du produit ne mentionne pas mais devrait (ex. : pratiques standard).
  - Le logiciel est difficile à comprendre, difficile à utiliser, lent, …

Sources des bogues

- Les erreurs dans les exigences logicielles et les documents de conception logicielle sont plus fréquentes que les erreurs dans le code source lui-même (Computer Finance Magazine)



Origin of Software Defects
(Source: Crosstalk, the Journal of Defense Software Engineering)

Objectifs des tests logiciels

Objectifs des tests logiciels

- S'assurer que toutes les fonctionnalités (demandées par le client) sont implémentées
- Détection et suppression des erreurs
- Déterminer le niveau de fiabilité du logiciel
- Gagner confiance dans le logiciel

Définitions des tests

- **Hetzel**: Toute activité visant à évaluer un attribut ou une capacité d'un programme ou d'un système. C'est la mesure de la qualité logicielle.
- **Beizer**: L'acte d'exécuter des tests. Les tests sont conçus puis exécutés pour démontrer la correspondance entre un élément et sa spécification.
- **Myers**: Le processus d'exécution d'un programme dans le but de détecter des erreurs.
- **IEEE**: Le processus d'exercice ou d'évaluation d'un système ou d'un composant système par des moyens manuels ou automatisés afin de vérifier qu'il satisfait les exigences spécifiées ou d'identifier les écarts entre les résultats attendus et réels.

Qu'est-ce que le test logiciel ?

- Le test est un processus technique, réalisé en expérimentant un produit logiciel dans un environnement contrôlé, en suivant une procédure spécifiée, dans le but d'observer une ou plusieurs caractéristiques du produit en démontrant la déviation de l'état réel du produit par rapport à l'état requis ou à la spécification.

- Les cas de test doivent être conçus pour couvrir un maximum de fautes tout en minimisant le temps et l'effort.

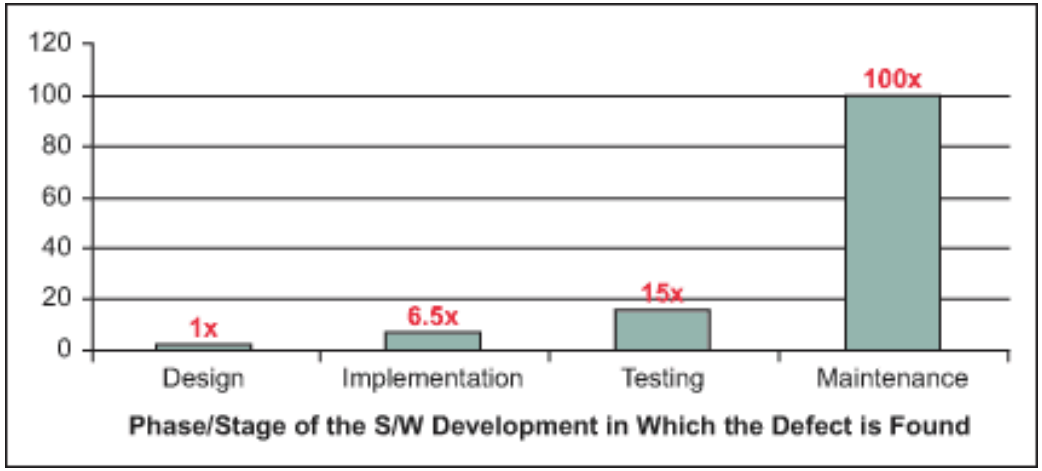
Coût des tests

Vous allez dépenser au moins la moitié de votre budget de développement en tests, que vous le vouliez ou non.

- Dans le monde réel, les tests sont l'activité principale après la conception.
- Restreindre les tests précoces entraîne généralement une augmentation des coûts.
- Une intégration approfondie matériel-logiciel nécessite davantage de tests.

Coût des tests : coût relatif des bugs

- Les bugs découverts plus tard coûtent plus cher à corriger
- Le coût de correction d'un bug augmente de façon exponentielle



Coûts relatifs de la correction des défauts logiciels (Source : IBM Systems Sciences Institute)

https://www.isixsigma.com/industries/software-it/defect-prevention-reducing-costs-and-enhancing-quality/

Coût de l'absence de tests

Les chefs de projet disent souvent que:
« Les tests sont trop coûteux. »

- Ne pas tester est encore plus coûteux.
- Planifier les tests après le développement est prohibitivement cher.
- Une station de test pour les cartes électroniques coûte **entre $30k et $150k** (station de test en production, automatisée, avec instrumentation).

- Les outils de test logiciels sont nettement moins coûteux

Pourquoi tester ?

Si vous ne savez pas pourquoi vous effectuez chaque test, il ne sera pas très utile.

- Les objectifs de test écrits et les exigences doivent être documentés.
- Quel fait chaque test essaie-t-il de vérifier ?
- Quels sont vos niveaux de couverture prévus ?
- Combien de tests sont suffisants ?
- Objectif courant – dépenser le budget … tester jusqu'à la date de livraison …
  - Appelé parfois "the **date criterion**"

Caractéristiques des tests logiciels

- Important pour contrôler la qualité du produit (et du processus).
- Souvent coûteux (Ne pas tester est encore plus coûteux).
- Difficile et stimulant.
- Critique en termes de temps

- **Cependant, les tests sont :**
  - **Non systématiques.**
  - **Effectués manuellement.**
  - **Sujet aux erreurs.**
  - **Peu valorisés :** « Si vous êtes un mauvais programmeur, vous pourriez devenir testeur » (**faux**).
  - **Destructifs.**

Tests pour l'amélioration et l'assurance qualité

- Les tests jouent un rôle crucial dans l'atteinte et l'évaluation de la qualité des produits logiciels.
  - Améliorer la qualité grâce au cycle: tester – détecter les défauts – corriger pendant le développement.
  - Évaluer la qualité globale du système durant les tests système, avant sa mise en service.

- Les tests sont à la fois un processus de vérification et un outil d'évaluation et d'amélioration de la qualité.

Types d'activités de test

- Les tests peuvent être divisés en quatre types généraux d'activités :
  1. Conception des tests
  2. Exécution des tests
  3. Évaluation des tests
  4. Automatisation des tests

- Chaque type d'activité nécessite des compétences différentes, des connaissances de base, une formation et une éducation adaptées.

- Aucune organisation de développement logiciel raisonnable n'utilise les mêmes personnes pour ces quatre activités.

Différentes activités dans le test des programmes (1)

- Un ingénieur de test effectue une séquence d'activités de test pour chaque cas de test.
- Les activités incluent l'identification des objectifs, la sélection des entrées, la configuration des environnements et l'analyse des résultats..

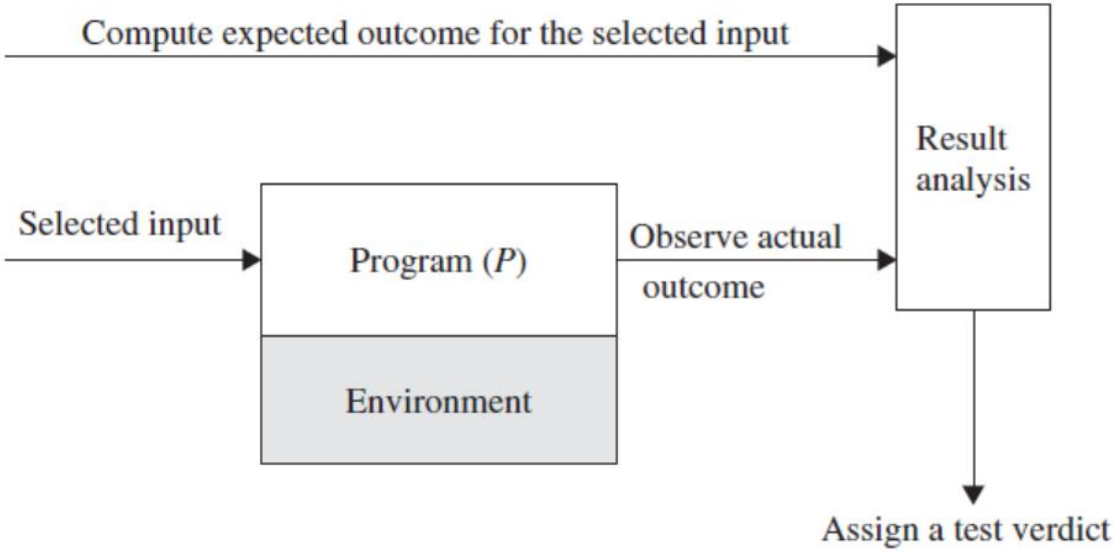


Fig 1.6, Page 46 (Naik, Kshirasagar, and Priyadarshi Tripathy. *Software testing and quality assurance: theory and practice*. John Wiley & Sons, 2011.)

Différentes activités dans le test des programmes (2)

1. **Identifier l'objectif :**
   - Définir le but du cas de test.
2. **Sélectionner les entrées :**
   - Choisir les entrées en fonction des exigences, du code source ou des attentes.
3. **Calculer le résultat attendu (Oracle de test) :**
   - S'appuyer sur une compréhension de haut niveau et les spécifications du programme pour déterminer les résultats attendus.
4. **Configurer l'environnement d'exécution :**
   - Vérifier que toutes les hypothèses externes sont satisfaites (ex. connexions réseau, bases de données).
   - Initialiser les systèmes distants si nécessaire (ex. démarrer un serveur pour des applications distribuées).

Différentes activités dans le test des programmes

5. Exécuter le programme :

- Exécuter le programme avec les entrées sélectionnées et observer les résultats réels.

6. Analyser les résultats des tests :

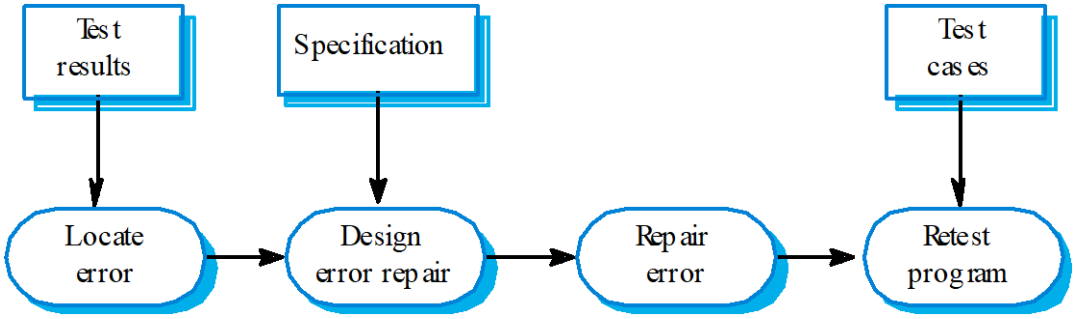
- Comparer les résultats réels aux résultats attendus.
- Attribuer un verdict :
  - **Succès :** Le résultat attendu est atteint.
  - **Échec :** Le résultat attendu n'est pas atteint.
  - **Inconclus :** Des tests supplémentaires sont nécessaires en raison de résultats ambigus (ex. : délais d'attente).

7. Rédiger un rapport de test :

- Documenter les étapes de reproduction des échecs, l'analyse et les détails pertinents.
- Inclure les entrées, les résultats attendus et l’environnement d’exécution pour plus de clarté.

Test vs. Débogage

- Test: Processus de recherche des entrées qui provoquent l'échec du logiciel (en exécutant dynamiquement un logiciel).
- Débogage: Processus de recherche d'un défaut après un échec.
- Le débogage implique :
  - Localiser le code source à l'origine du bug
  - Corriger le bug
- Le débogage se produit **APRÈS** les tests.
- Après le débogage, **PLUS** de tests sont nécessaires.



Test vs. Assurance Qualité (AQ)

- Le test vise à établir la confiance qu'un programme:
  - FAIT ce qu'il est censé faire.
  - NE FAIT PAS ce qu'il n'est pas censé faire.

Le test n'est pas l'Assurance Qualité (AQ) :

- Le test est nécessaire mais ne suffit pas pour le processus d'AQ.
  - Le test contribue à améliorer la qualité en aidant à identifier les problèmes.

- L'AQ établit des normes que les membres du projet (y compris les testeurs) doivent suivre **afin** de construire un meilleur logiciel.

Niveaux de Test Basés sur la Maturité du Processus de Test (Niveaux de Test de Beizer) (1)

Niveau 0 de Pensée

- Il n'y a aucune différence entre test et débogage.
- Ne fait pas la distinction entre un comportement incorrect et des erreurs dans le programme.
- Ne contribue pas au développement d'un logiciel fiable ou sécurisé.

Ammann, Paul, and Jeff Offutt. Introduction to software testing. Cambridge University Press, 2017.

Niveaux de Test Basés sur la Maturité du Processus de Test (Niveaux de Test de Beizer) (2)

Niveau 1 de Pensée

- Le but du test est de prouver la « Correctness ».
- La « correctness » est impossible à atteindre.
- Que dire s'il n'y a aucune défaillance ?
  - Bon logiciel ou mauvais tests ?
- Les ingénieurs en test n'ont pas :
  - D'objectifs stricts
  - De véritable règles d'arrêt
  - De techniques de tests formelles
  - Les gestionnaires de tests sont impuissants

Ammann, Paul, and Jeff Offutt. Introduction to software testing. Cambridge University Press, 2017.

Niveaux de Test Basés sur la Maturité du Processus de Test (Niveaux de Test de Beizer) (3)

Niveau 2 de Pensée

- Le but du test est de montrer que **le logiciel ne fonctionne pas.**
- L'objectif est de démontrer les <u>échecs</u>.
- Rechercher des échecs est une activité <u>négative</u>.
- Que faire s'il n'y a <u>aucun échec</u> ?

Ammann, Paul, and Jeff Offutt. Introduction to software testing. Cambridge University Press, 2017.

Niveaux de Test Basés sur la Maturité du Processus de Test (Niveaux de Test de Beizer) (4)

- Place les testeurs et les développeurs dans une relation conflictuelle

Niveaux de Test Basés sur la Maturité du Processus de Test (Niveaux de Test de Beizer) (5)

Niveau 3 de Pensée

- Le but des tests n'est pas de prouver quoi que ce soit de spécifique, mais de **réduire le risque lié à l'utilisation du logiciel.**
- Chaque fois que nous utilisons un logiciel, nous encourons un certain risque.
- Le risque peut être faible et les conséquences sans importance.
- Le risque peut être élevé et les conséquences catastrophiques.
- Les testeurs et les développeurs travaillent ensemble pour réduire ces risques.

Ammann, Paul, and Jeff Offutt. Introduction to software testing. Cambridge University Press, 2017.

Niveaux de Test Basés sur la Maturité du Processus de Test (Niveaux de Test de Beizer) (6)

Niveau 4 de Pensée

- Les tests sont une discipline mentale qui aide tous les professionnels de l'informatique à développer des logiciels de meilleure qualité
- Les tests sont un moyen d'améliorer la qualité
- Les ingénieurs de test peuvent devenir les leaders techniques du projet.
- La principale responsabilité est de mesurer et d'améliorer la qualité des logiciels
- Leur expertise devrait aider les développeurs

Ammann, Paul, and Jeff Offutt. Introduction to software testing. Cambridge University Press, 2017.

Types de tests logiciels (selon la phase du cycle de vie)

- **Tests unitaires :** se concentrent sur les tests d'unités ou de composants individuels de manière isolée pour garantir leur bon fonctionnement ; ils peuvent impliquer des techniques de boîte blanche, de boîte noire ou les deux.

- **Tests d'intégration :** garantissent qu'un ensemble de composants ou de sous-systèmes intégrés fonctionne comme prévu.

- **Tests système :** vérifient que le système complet répond à ses exigences et que toutes les fonctionnalités souhaitées fonctionnent comme prévu.

- **Tests de régression** : garantissent que les modifications récentes du code n'ont pas d'impact négatif sur les fonctionnalités existantes du système, souvent à l'aide d'un sous-ensemble de cas de test système.

- **Tests bêta :** réalisés par des clients sélectionnés pour évaluer le système avant sa sortie officielle.

- **Tests d'acceptation :** réalisés par le client contractant pour décider si l'application répond à ses exigences avant l'achat ou le déploiement.

Phase	Technique
Coding	Unit testing
Integration	Integration testing
System integration	System testing
Maintenance	Regression testing
Post system, pre-release	Beta-testing

Table 1.5 from Mathur, Aditya P. Foundations of software testing, 2/e. Pearson Education India, 2013.

Types de tests logiciels (orientés vers des objectifs)

Goal	Testing Technique	Example
Advertised features	Functional	
Security	Security	
Invalid inputs	Robustness	
Vulnerabilities	Vulnerability	Penetration testing
Errors in GUI	GUI	Capture/playback Event sequence graphs Complete Interaction Sequence
Operational correctness	Operational	Transactional-flow
Reliability assessment	Reliability	
Resistance to penetration	Penetration	
System performance	Performance	Stress testing
Customer acceptability	Acceptance	
Business compatibility	Compatibility	Interface testing Installation testing
Peripherals compatibility	Configuration	
Foreign language compatibility	Foreign language	

Table 1.6 from Mathur, Aditya P. Foundations of software testing, 2/e. Pearson Education India, 2013.

- **Les tests de robustesse** évaluent la robustesse d'une application face à des entrées **non intentionnelles ou non valides.**
- **Tests de pénétration** : évalue l'efficacité des politiques et mesures de sécurité d'une organisation en simulant des tentatives d'accès non autorisées, en utilisant à la fois des techniques de boîte noire et de boîte blanche.
- **Les tests de stress** testent le comportement d'une application sous contrainte, comme un grand nombre d'entrées ou de requêtes sur un serveur Web.
- **Les tests de performance** sont utilisés pour tester si une application répond ou non à ses exigences de performance, par exemple, **le traitement des requêtes en ligne dans un laps de temps donné**.

Tests dans différents processus SDLC

Process	Attributes
Testing in waterfall model	Usually done toward the end of the development cycle.
Testing in V model	Explicitly specifies testing activities in each phase of the development cycle.
Spiral testing	Applied to software increments, each increment might be a prototype that eventually leads to the application delivered to the customer. Proposed for evolutionary software development.
Agile testing	Used in agile development methodologies such as extreme programming (XP).
Test Driven Development (TDD)	Requirements specified as tests.

Table 1.8 from Mathur, Aditya P. Foundations of software testing, 2/e. Pearson Education India, 2013.

Tests dans le cadre du SDLC traditionnel : modèle en cascade

- Se produit principalement vers la fin du processus de développement.
- La vérification et la validation sont effectuées sur les documents produits à chaque phase, en tant qu'activités essentielles.
- Les défauts introduits dans les phases précédentes et découverts ultérieurement peuvent être coûteux à corriger en raison de la nature séquentielle du modèle.

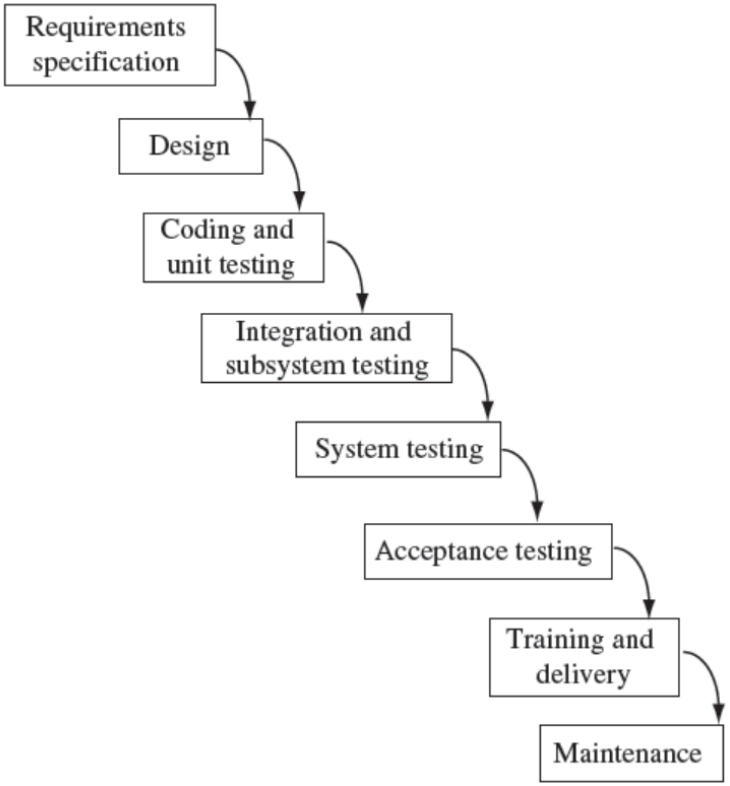


Fig 1.15 from Mathur, Aditya P. Foundations of software testing, 2/e. Pearson Education India, 2013.

Tests dans le SDLC traditionnel : modèle en V

- Le modèle en V conserve les mêmes phases de développement que le modèle en cascade mais met l'accent sur l'intégration des activités de test.

- La conception des tests commence dès que les exigences sont disponibles, ce qui permet une planification et une préparation précoces de la validation.

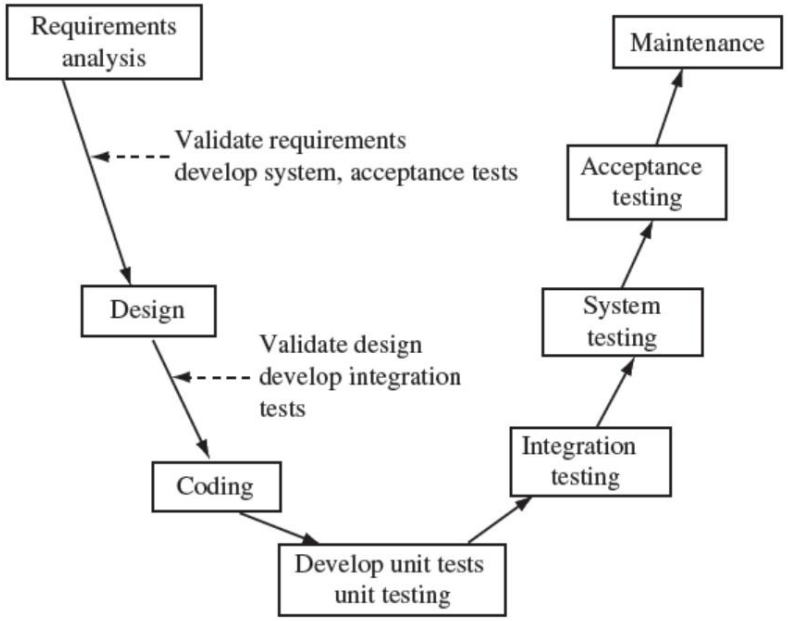


Fig 1.16 from Mathur, Aditya P. Foundations of software testing, 2/e. Pearson Education India, 2013.

Tests dans le cadre du SDLC traditionnel : tests en spirale

- Processus de développement incrémentiel.
- La complexité et la concentration des activités de test augmentent à chaque itération du prototype en évolution.
  - **Premières étapes** : planification des tests
  - **Itérations suivantes** : réalisation de tests unitaires et d'intégration à mesure que le prototype évolue
  - **Étape finale** : mise en avant des tests système et d'acceptation lorsque les exigences sont bien définies.
- Le coût des activités de test augmente avec les itérations suivantes

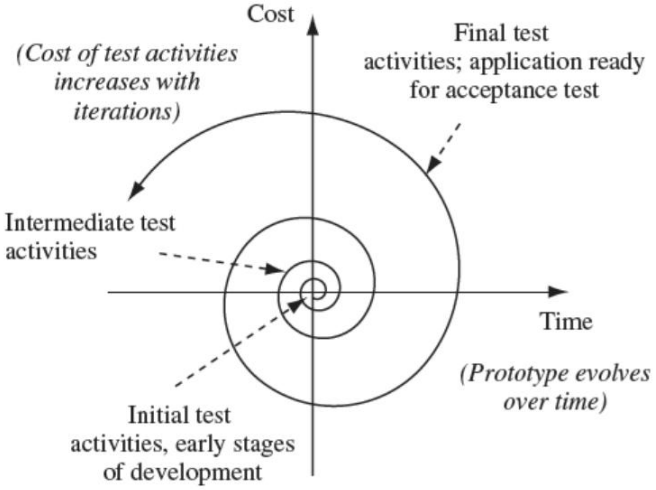


Fig 1.17 from Mathur, Aditya P. Foundations of software testing, 2/e. Pearson Education India, 2013.

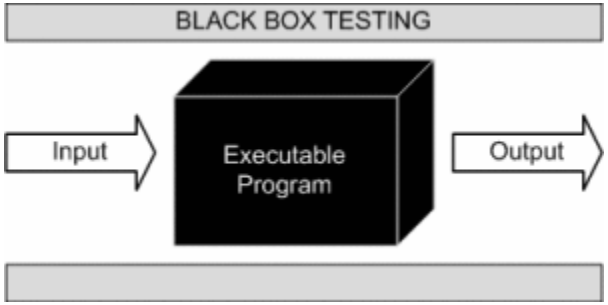
Tests Agiles

- Intégration des activités de test tout au long du processus de développement.

- Les tests agiles mettent l'accent sur la collaboration entre les testeurs, les développeurs et les clients.
  - Les testeurs et les développeurs travaillent ensemble en tant que partenaires plutôt qu'en tant qu'adversaires.

- Les tests sont effectués souvent et en petites parties gérables pour garantir un retour d'information continu et une amélioration progressive.

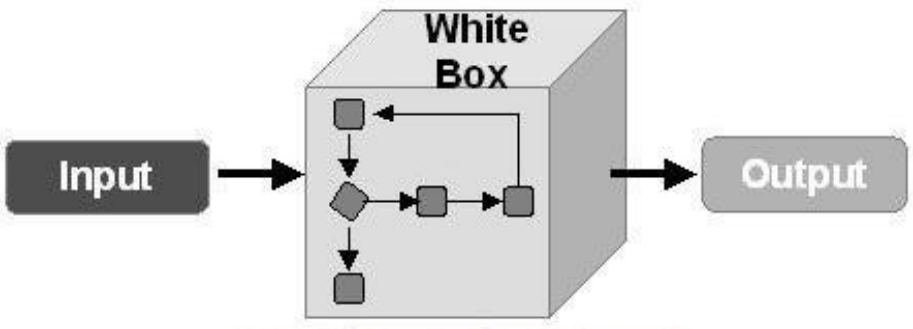
Types de tests logiciels : boîte noire ou boîte blanche (1)



- Les cas de test sont dérivés de la spécification formelle du système.
- La sélection des cas de test peut être effectuée sans aucune référence à la conception ou au code du programme.
- Ne teste que la fonctionnalité et les caractéristiques du programme.
  - Pas le fonctionnement interne.
- Également connu sous le nom de test fonctionnel.
- **Avantages:**
  - La sélection des cas de test est effectuée avant la mise en œuvre d'un programme.
  - Aide à obtenir une conception et un codage corrects par rapport à la spécification.

Types de tests logiciels : boîte noire ou boîte blanche (2)

- Les cas de test sont dérivés de la structure ou du code réel du programme.
- Également appelés tests structurels ou tests en boîte de verre.



- **Avantages:**
  - Teste les détails internes du code ;
  - Vérifie tous les chemins qu'un programme peut exécuter.
- **Limitations:**
  - Attendez d'avoir conçu et codé le programme testé pour sélectionner les cas de test.

Défaut, erreur ou échec (taxonomie des tests par Ammann et Offutt)

- **Défaut logiciel (fault) :**

Défaut statique dans le logiciel

- **Erreur logicielle (error) :**

État interne incorrect qui est la manifestation d'un défaut

- **Défaillance (failure):**

Comportement externe incorrect par rapport aux exigences ou à une autre description du comportement attendu

Ammann, Paul, and Jeff Offutt. Introduction to software testing. Cambridge University Press, 2017.

Exemple

- Supposons un programme qui utilise la plus grande valeur résultant d'une fonction d'addition et d'une fonction carrée

```
//…code……
r1 = add (a , b);
r2 = square (x) ;
//uses the larger of r1 and r2
//……rest of the code…
```

Exemple (suit)

```
int square (int x){
 if (x == 0 )
  return 0;
  else
   return x*2; ⟵ fault
}
int add (int a, int b){
 return a+b;
}
```

- L'erreur ici est que le carré de x a été calculé comme x*2 au lieu de x*x.
- Un cas de test où l'erreur ne sera pas exécutée est lorsque x = 0.
- Un cas de test où l'erreur sera exécutée mais n'entraînera pas d'erreur est lorsque x = 2.

Exemple (suit)

```
int square (int x){
 if (x == 0 )
  return 0;
  else
   return x*2; ⟵ fault
}
int add (int a, int b){
 return a+b;
}
```

- Un cas de test dans lequel l'erreur sera exécutée et entraînera une erreur qui n'entraîne PAS d'échec est lorsque, par exemple, x=3 et a et b=20.

- Un cas de test dans lequel une erreur entraînera un échec est lorsque, par exemple, x= 3 et a et b = 4.

Validation et vérification (V&V)

- La vérification et la validation interviennent à chaque phase du cycle de vie du développement logiciel.
  - Exigences
  - Conception
  - Code
- Objectif ultime
  - Le logiciel est « adapté à l'usage ».
- Le logiciel n'est pas nécessairement exempt de défauts à 100 %.
- Il doit plutôt être **suffisamment bon**
  - pour l'usage auquel il est destiné ; et
  - le type d'utilisation déterminera le degré de confiance nécessaire.

Validation

Créons-nous le bon produit ?

(Latin validus - sain, solide, efficace)

- Le processus d'évaluation d'un système pendant et à la fin du processus de développement pour garantir la conformité avec l'utilisation prévue (IEEE)
- Le logiciel doit faire ce dont l'utilisateur a réellement besoin.
- La validation dépend généralement de la connaissance du domaine, c'est-à-dire de la connaissance de l'application pour laquelle le logiciel est écrit.

Par exemple, la validation d'un « avion » nécessite des **connaissances de la part des ingénieurs** et des pilotes de l'aérospatiale.

Vérification

Est-ce que nous construisons le produit correctement ?

(latin veritas - vérité ou intégrité)

- Le processus de détermination de la conformité des produits d'une phase donnée du processus de développement logiciel aux exigences établies lors de la phase précédente (IEEE)

- La vérification est généralement une **activité plus technique** qui utilise les connaissances sur les artefacts, les exigences et les spécifications des logiciels individuels

V&V indépendant

- Processus par lequel la V&V est réalisée par une organisation qui n'est ni le développeur ni l'acquéreur du logiciel

- Trois types d'indépendance :

1. Indépendance managériale
   - Décision séparée sur les domaines du logiciel à tester et les techniques à utiliser
   - Détermination du calendrier des tâches
2. Indépendance financière
   - Les coûts de V&V sont financés séparément
   - Aucun risque de détournement de ressources pouvant entraîner des retards
3. Indépendance technique
   - Différente des développeurs ou des analystes
   - Utilisation d'outils différents

Confiance aux V & V

- Cela dépend de l'objectif du système, des attentes des utilisateurs et de l'environnement marketing

  - Fonction du logiciel
  - Le niveau de confiance dépend de l'importance du logiciel pour une organisation.

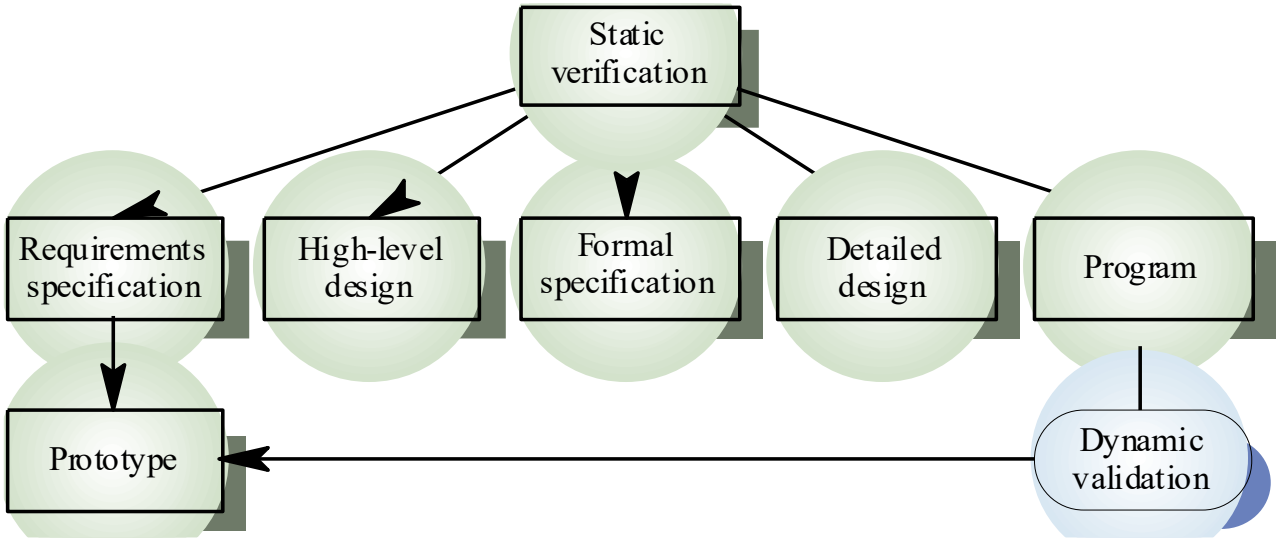
  - Attentes des utilisateurs
  - Les utilisateurs peuvent avoir de faibles attentes envers certains types de logiciels.

  - Environnement marketing
  - Il peut être plus important de mettre un produit sur le marché le plus tôt possible que de trouver des défauts dans le programme.

Approches V&V

Dans le cadre du processus V&V, il existe deux approches complémentaires de l'analyse du système :

- Inspections, revues et walkthrough (analyse statique)
- Tests du logiciel : Tests en exécutant le programme avec des entrées réelles et en observant son comportement (analyse dynamique)



Analyse statique

- Tests **sans exécution du programme**
  - Concerne l'analyse de la représentation statique du système pour découvrir les problèmes.
  - Les « walkthroughs, revues et inspections » : formes de tests humains qui impliquent que les gens travaillent ensemble en coopération.
  - Examiner les documents et le système logiciel pendant les différentes phases du cycle de vie du développement.
  - Généralement plus rentable que les tests de détection des défauts au niveau de l'unité et du module. **Plus de 60 % des erreurs de programme peuvent être détectées par des inspections informelles du programme.**
  - Permet de combiner la détection des défauts avec d'autres contrôles de qualité.
- Complément par une analyse de code et de documents basée sur des outils.

Exemples de Vérifications d'analyse statique du code

Erreurs de données

- Variables utilisées avant l'initialisation
- Variables déclarées mais jamais utilisées
- Variables affectées deux fois mais jamais utilisées entre les affectations
- Variable utilisée là où une constante fonctionnerait mieux (variable non mise à jour).
- Une variable doit-elle être déclarée comme une chaîne plutôt qu'un tableau de caractères ?
- Y a-t-il des variables avec des noms similaires ?
- Une variable affectée d'une valeur d'un type différent

Exemples de Vérifications d'analyse statique du code

Erreurs de données

- Violations possibles des limites du tableau

```
int arr[] = {1,2,3,4,5};
printf("arr [0] is %d\n", arr[0]);

// arr[10] is out of bound
printf("arr[10] is %d\n", arr[10]);
return 0;
```

Output :

```
arr [0] is 1
arr[10] is -1786647872
```

Garbage value

Peut également conduire à :

- Une erreur d'exécution/un crash
- Une erreur de segmentation

Exemples de Vérifications d'analyse statique du code

Erreurs de calcul

- Est-ce que des calculs utilisant des variables ont des types de données différents ?
  - *Exemple* : ajouter un nombre à virgule flottante à un entier
- Est-ce que des calculs utilisant des variables ont le même type de données mais sont de tailles différentes ?
  - Exemple : dans un programme C, ajouter un entier long (32 bits) à un entier court (16 bits)
- Les règles de conversion du compilateur pour les variables de type ou de taille incohérentes sont-elles comprises ?
- Est-il possible d'avoir une division par 0 ?
- La valeur d'une variable peut-elle sortir de sa plage de valeurs significatives ?
  - *Exemple* : une probabilité peut-elle être inférieure à 0 ou supérieure à 1.
- Les parenthèses sont-elles nécessaires pour clarifier les règles de priorité des opérateurs ?

Exemples de Vérifications d'analyse statique du code

Défauts de contrôle

- Code inaccessible (code mort)
- Fonctions non appelées
- Les boucles se terminent-elles ? Si ce n'est pas le cas, est-ce intentionnel ?
- Chaque instruction switch possède-t-elle une clause default ?
- Y a-t-il des instructions switch imbriquées dans des boucles ?
  - **Soyez prudent**, car les instructions break dans les instructions switch ne sortiront pas de la boucle… mais les instructions break qui ne sont pas dans les instructions switch sortiront de la boucle.
- Est-il possible qu'une boucle ne s'exécute jamais ?

Exemples de Vérifications d'analyse statique du code

Défauts d'entrée/sortie

- Si le fichier ou le périphérique n'est pas prêt, cette condition d'erreur est-elle gérée ?
- Le logiciel gère-t-il la situation de déconnexion du périphérique externe ?
- Tous les messages d'erreur ont-ils été vérifiés quant à leur exactitude, leur pertinence, leur grammaire et leur orthographe ?
- Toutes les exceptions sont-elles gérées par une partie du code ?
- Le logiciel respecte-t-il le format spécifié de la date lue ou écrite sur le périphérique externe ?

Exemples de Vérifications d'analyse statique du code

Défauts de paramètres de sous-routine

- Si des constantes sont transmises à la sous-routine en tant qu'arguments, sont-elles modifiées accidentellement dans la sous-routine ?

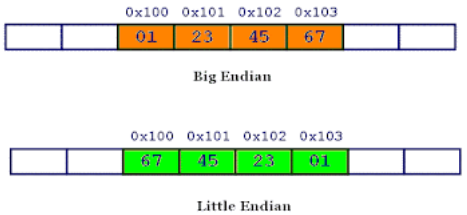
- Les unités de chaque paramètre correspondent-elles aux unités de chaque argument correspondant ?
  - Par exemple, système anglais ou système métrique

Exemples de Vérifications d'analyse statique du code

Autres types de verifications

- Le code prend-il en charge l'ASCII et l'Unicode ?
- Qu'en est-il des problèmes d'internationalisation ?
- Votre code repose-t-il sur des API obsolètes ?
- Votre code est-il portable sur d'autres plateformes OS ? (Portabilité)
- Votre code peut-il être porté vers des architectures avec des ordres d'octets différents ? *Exemple* : Little-endian : l'octet de poids faible (LSB) est stocké à l'adresse mémoire la plus basse, tandis que big-endian : l'octet de poids fort (MSB) est stocké à l'adresse mémoire la plus basse, comme on écrit généralement les nombres (chiffres les plus significatifs en premier).

X = 0x01234567



Static Analysis: Walkthroughs

- Généralement effectué en une seule réunion.
- Évaluer un produit logiciel pour
  - trouver des anomalies et améliorer le produit logiciel.
  - Envisager des implémentations alternatives (à un niveau bas et détaillé).
  - Évaluer la conformité aux normes et aux spécifications.
- **Assez informel.**
- Aucune formation formelle n'est requise au préalable.
- Le succès dépend de **l'expérience et des compétences des membres de l'équipe**.
- **Peut être effectué à n'importe quelle phase du processus de développement logiciel et sur n'importe quel artefact (SRS, diagrammes de cas d'utilisation, diagrammes de classes, code, etc.)**

Analyse Statique : Inspection Logicielle

- Approche formelle basée sur une **checklist** pour détecter les erreurs.
- Destinée explicitement à la détection des défauts **(et non à la correction).**
- Les défauts peuvent être des erreurs logiques et des anomalies dans le code.
- Les membres de l'équipe nécessitent une formation formelle préalable.
- Son succès dépend de:
  - La bonne application du processus d'inspection,
  - Les vérifications appliquées,
  - La rigueur des inspecteurs.
- Peut être effectuée à n'importe quelle phase du processus de développement logiciel et sur n'importe quel artefact (SRS, diagrammes UC, diagrammes de classes, code, etc.).

Exemple : Liste de Vérification de l'Inspection du Code

1. **Défauts de Déclaration des Variables, Attributs et Constantes**
   - Les noms des variables et constantes sont-ils descriptifs et conformes aux conventions de nommage ?
   - Y a-t-il des variables ou des attributs aux noms trop similaires et pouvant prêter à confusion ?
   - Chaque variable et chaque attribut est-il correctement initialisé ?
   - Certaines variables non locales pourraient-elles être rendues locales ?
   - Y a-t-il des constantes littérales qui devraient être nommées comme constantes ?
   - Y a-t-il des variables ou des attributs qui devraient être des constantes ?
   - Y a-t-il des attributs qui devraient être des variables locales ?
   - Tous les attributs ont-ils les modificateurs d'accès appropriés (privé, protégé, public) ?
   - Y a-t-il des attributs statiques qui devraient être non statiques, ou vice-versa ?

2. **Défauts dans la Définition des Méthodes**
   - Les noms des méthodes sont-ils descriptifs et conformes aux conventions de nommage ?
   - Toutes les méthodes ont-elles les modificateurs d'accès appropriés (privé, protégé, public) ?
   - Y a-t-il des méthodes statiques qui devraient être non statiques, ou vice-versa ?

3. **Défauts dans la Définition des Classes**
   - Chaque classe possède-t-elle des constructeurs appropriés ?
   - Certaines sous-classes ont-elles des membres communs qui devraient être placés dans la super-classe ?

Exemple : Liste de Vérification de l'Inspection du Code

4. Défauts de Calcul/Numériques

- Y a-t-il des calculs impliquant des types de données mixtes ?
- Les parenthèses sont-elles utilisées pour éviter toute ambiguïté ?

5. Défauts de Commentaires

- Chaque méthode, classe et fichier possède-t-il un commentaire d'en-tête approprié ?
- Chaque déclaration d'attribut, de variable et de constante a-t-elle un commentaire ?
- Le comportement sous-jacent de chaque méthode et classe est-il exprimé en langage simple ?
- Le commentaire d'en-tête pour chaque méthode et classe est-il cohérent avec leur comportement ?
- Les commentaires et le code sont-ils en accord ?
- Les commentaires aident-ils à la compréhension du code ?
- Y a-t-il suffisamment de commentaires dans le code ?
- Y a-t-il trop de commentaires dans le code ?

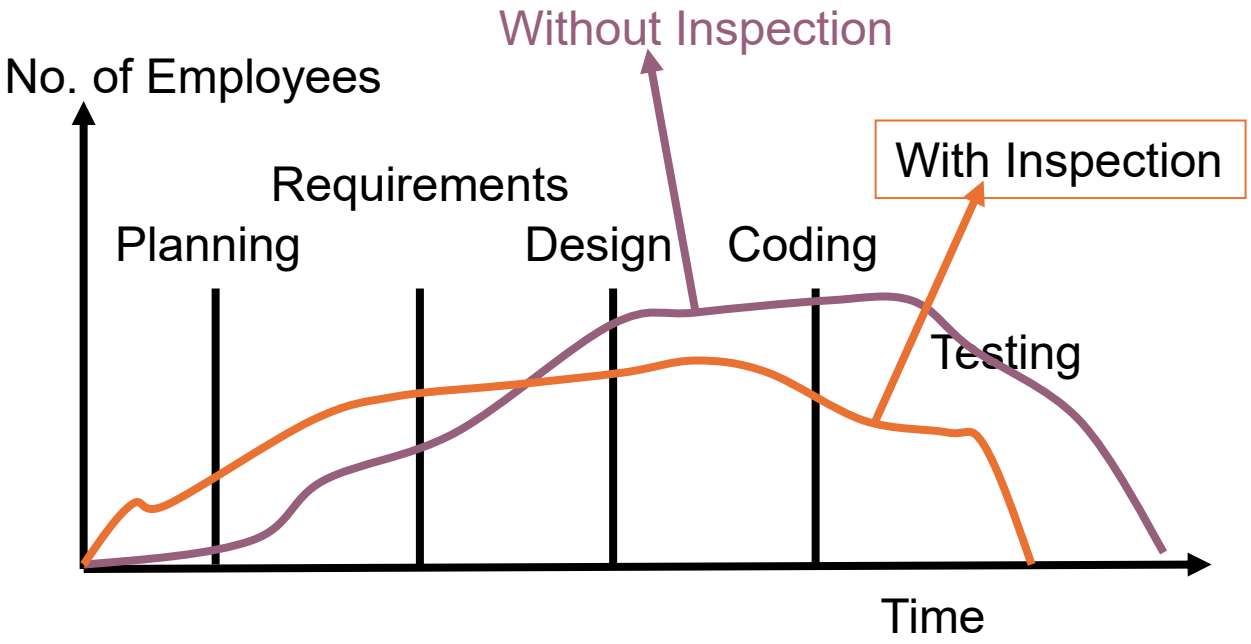
6. Défauts de Mise en Page et d'Emballage

- Une indentation et un format de mise en page standard sont-ils utilisés de manière cohérente ?
- Pour chaque méthode : ne dépasse-t-elle pas environ 60 lignes ?

7. Défauts de Modularité (MO)

- Les bibliothèques de classes Java/C# sont-elles utilisées à bon escient et au bon moment ?
- Y a-t-il des bibliothèques importées mais non utilisées dans une classe donnée ?

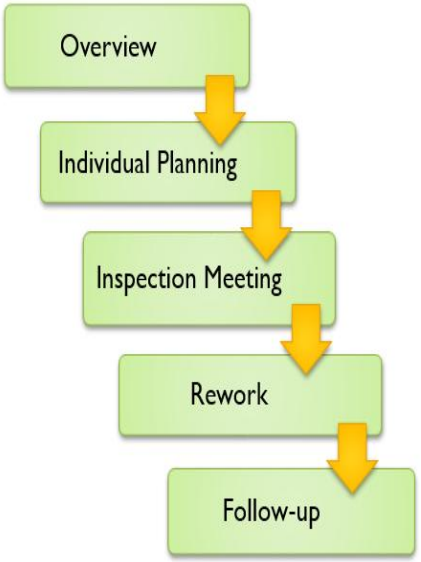
Coût d'inspection du logiciel



Conditions préalables à l'inspection

- **Une spécification/un code** précis doit être disponible (en cas d'implémentation, un code **syntaxiquement correct** doit être disponible)
- Les membres de l'équipe de test statique doivent être **familiarisés avec les normes de l'organisation**
- Une liste de **liste de contrôle des erreurs** doit être préparée
- La direction doit accepter que l'inspection augmentera les coûts dès le début du processus logiciel

Le processus d'inspection



1. Aperçu (équipe entière)
   - Qu'est-ce qui sera inspecté ?
   - Pourquoi passons-nous du temps à inspecter un tel artefact ?
   - Désignation des rôles de l'équipe.
2. Préparation (individuelle)
   - Le code et les documents associés sont distribués à l'avance à l'équipe d'inspection
   - listes de contrôle des indices sur la recherche d'erreurs
3. Réunion d'inspection (équipe entière)
   - un « lecteur » est choisi par le modérateur
   - L'inspection a lieu et les erreurs découvertes sont notées
   - le modérateur prépare un rapport écrit dans un délai d'un jour
4. Reprise (propriétaire) : des modifications sont apportées pour réparer les erreurs découvertes
5. Suivi (modérateur)
   - si > 5 % du matériel a été retravaillé, l'élément entier est réinspecté

Inspection du code modifié

- « Étant donné que la plupart des modifications sont minimes... elles sont souvent considérées à tort comme trivialement simples et traitées en conséquence ; ... Cependant, **toutes les modifications méritent d'être inspectées**... »

- « La tendance humaine est de considérer que la « correction » d'un problème est elle-même exempte d'erreurs. ... Le nombre de mauvaises corrections peut être réduit par une simple inspection après une compilation propre de la correction. »

Inspections vs. Walkthrough

Propriétés	Inspections	Walkthrough
Formation formelle du modérateur	Oui	Non
Rôles définis des participants	Oui	Non
Qui « dirige » le processus	Modérateur	Propriétaire
Utilisation de listes de contrôle ?	Oui	Non
Suivi formel	Oui	Non
Niveau de rigueur	Formel	Informel

Inspections et walkthrough vs. Revues

- Les inspections et les walkthroughs se concentrent sur **l'évaluation de l'exactitude**.
- Une revue est aussi un **processus informel (aucune formation formelle préalable).**
  - Son succès dépend des compétences et de l'expérience des réviseurs.
- L'équipe de revue se préoccupe davantage des écarts par rapport au **modèle conceptuel** et aux exigences.
  - Les revues ne se **concentrent pas sur la découverte de défauts techniques** mais sur le fait de s'assurer que la conception et le développement **répondent avec précision aux besoins de l'application.**
  - Les revues visent à vérifier que **des niveaux de qualité acceptables sont atteints**.

Question

- Quelle technique relève davantage d'un processus de **validation**, et laquelle d'un processus de **vérification** ?

  - **Walkthrough** — **Vérification**
  - **Inspections** — **Vérification**
  - **Revues** — **Validation**

Métriques d'Inspection

De nombreuses métriques peuvent être calculées lors d'un processus d'inspection :

- **Total des défauts trouvés : A + B – C**

A : défauts trouvés par l'inspecteur 1,

B : défauts trouvés par l'inspecteur 2,

C : défauts communs trouvés par 1 et 2

- Le nombre de défauts majeurs et mineurs trouvés
- Le rapport du nombre de défauts majeurs par rapport au total des défauts trouvés
- **Densité des défauts** : Total des défauts trouvés / Taille

  Taille = total number of pages or lines of code or some other measure
- **Taux d'inspection :**
  - Taille / Temps total d'inspection (somme du temps de tous les réviseurs plus le temps total passé en réunion). Exemple: 15 pages/heure.
- **Taux de détection des défauts :**
  - **Total des défauts trouvés / Temps total d'inspection**

Cas de Test et Suites de Test

- Tous les types de tests se résument à la création de cas de test et à leur exécution.

- Cas de Test : est composé des **valeurs du cas de test, résultats attendus, valeurs préfixes, valeurs** suffixes nécessaires pour une exécution complète et une évaluation du logiciel testé (SUT).

- Suite de Test : est simplement un ensemble de cas de test.

Composants d'un cas de test

Valeurs de Préfixe : Toute entrée nécessaire pour mettre le logiciel dans l'état approprié afin de recevoir les valeurs du cas de test.

Valeurs du Cas de Test : Les valeurs d'entrée nécessaires pour compléter une exécution du logiciel testé.

Valeurs de Postfixe : Toute entrée qui doit être envoyée au logiciel après les valeurs des cas de test. Il existe deux types de valeurs de postfixe :

- **Valeurs de Vérification** : Valeurs nécessaires pour voir les résultats des valeurs du cas de test.
- **Commandes de Sortie :** Valeurs nécessaires pour terminer le programme ou le ramener à un état stable.

Cas de Test : Vérifier la fonctionnalité de login réussie d'un site web

1. Valeurs de Préfixe (État d'Initialisation) :

- Ouvrir le site e-commerce.
- Naviguer vers la page de login.
- S'assurer que le site est en ligne et fonctionnel.

2. Valeurs du Cas de Test (Entrée et Exécution) :

- Saisir les identifiants suivants:
  - Nom d'utilisateur : valid_user
  - Mot de passe : valid_password
  - Cliquer sur le bouton de connexion.

3. Résultats du Cas de Test :

- Confirmer que l'utilisateur est redirigé vers la page d'accueil.
- Vérifier que le nom de l'utilisateur (ex. : "Bienvenue, valid_user") apparaît à l'écran.

4. Valeurs de Postfixe :

- Valeurs de Vérification :
  - Vérifier que le jeton de session est stocké (ex. : dans les cookies ou le stockage local).
- Valeurs de Sortie :
  - Fermer la fenêtre.

Cas de Test : Créer et Vérifier une Nouvelle Interface avec une Adresse IPv4 sur un Routeur

1. **Valeurs de Préfixe (État d'Initialisation) :**
   - Se connecter au routeur à l'aide d'un terminal (ex. : via SSH ou câble console).
   - S'assurer que la session terminal est active et en mode EXEC privilégié (invite Router#).
   - Vérifier que le routeur possède une interface inutilisée pour la configuration (ex. : GigabitEthernet0/1).
   - Entrer en mode de configuration globale :
     - Router# configure terminal

2. **Valeurs du Cas de Test (Exécution) :**
   - Sélectionner l'interface à configurer :
     - Router(config)# interface GigabitEthernet0/1
   - Attribuer la nouvelle adresse IPv4 et le masque de sous-réseau :
     - Router(config-if)# ip address 192.168.1.1 255.255.255.0
   - Activer l'interface :
     - Router(config-if)# no shutdown

Cas de Test : Créer et Vérifier une Nouvelle Interface avec une Adresse IPv4 sur un Routeur

3. Valeurs de Postfixe :

- Verification Values:
  - Exit to privileged EXEC mode: Router(config-if)# exit
  - Run the show command to verify the IP assignment: Router# show ip interface brief
  - Confirm the `GigabitEthernet0/1` interface has the assigned IP address (`192.168.1.1`) and is in an "up/up" state.

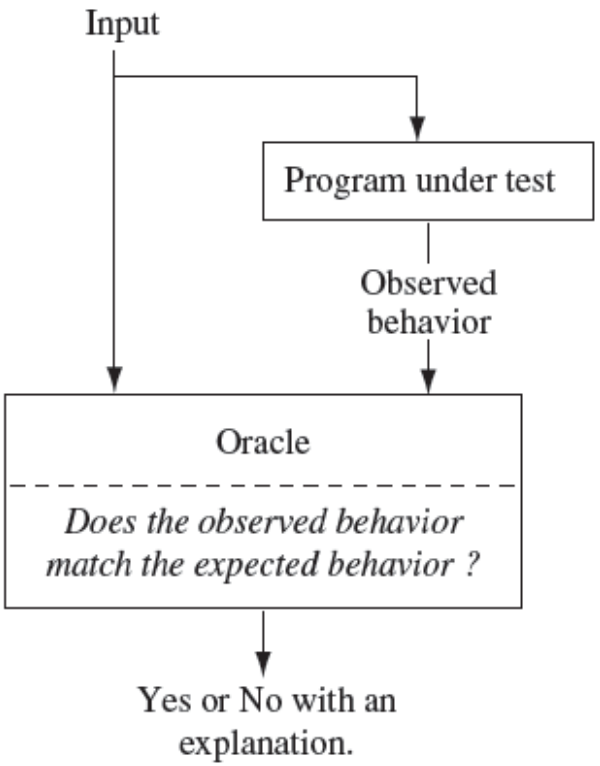
```
Router# show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     unassigned      YES unset  administratively down down
GigabitEthernet0/1     192.168.1.1     YES manual up                    up
GigabitEthernet0/2     10.1.1.1        YES manual up                    up
Vlan1                  192.168.100.1   YES manual up                    up
```

- Valeurs de Sortie :
  - Déconnecter du routeur : exit

Oracle de Test (1)

- Un oracle de test utilise des résultats/comportements attendus pour déterminer si un test a réussi ou échoué.
- Un oracle peut être :
  - **Un être humain** : Par exemple, on pourrait calculer l'inverse d'une matrice à la main et vérifier si le résultat produit par le programme correspond aux résultats du calcul manuel.

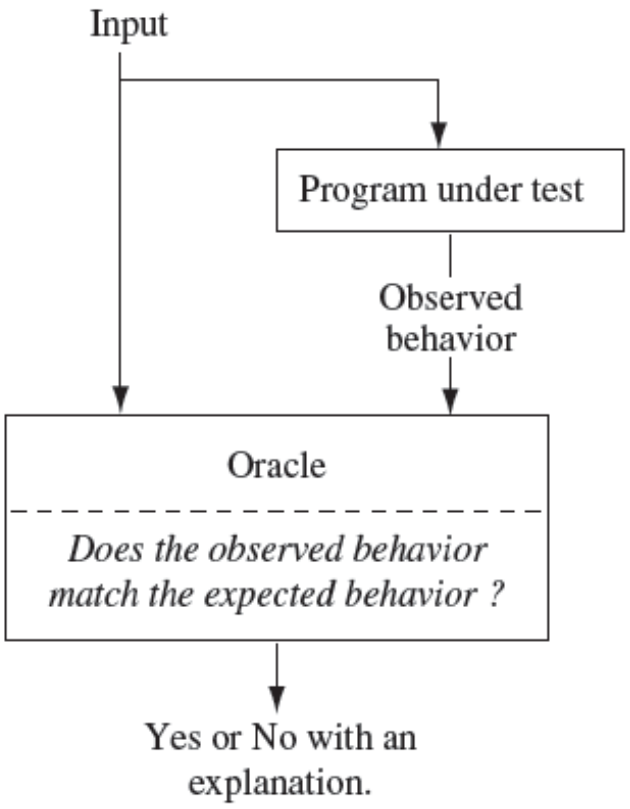
  Or
  - **Un programme** : Par exemple, on pourrait utiliser un programme de multiplication de matrices pour vérifier si un programme d'inversion de matrices a produit le bon résultat. Dans ce cas, le programme d'inversion de matrices inverse une matrice donnée A et génère B comme matrice de sortie. Le programme de multiplication peut vérifier si $A \times B = I$, en tenant compte d'une certaine marge d'erreur sur les éléments de la matrice identité I.



Oracle de Test (2)

- Un oracle basé sur un humain est sujet aux erreurs, plus lent que l'exécution d'un programme et limité à des cas triviaux. Pourtant, malgré ces inconvénients, il est souvent le meilleur oracle disponible.

- Utiliser des programmes comme oracles offre rapidité, précision et capacité à vérifier des calculs complexes, comme les opérations sur de grandes matrices, de manière plus efficace que les oracles humains.



Testabilité des logiciels

- La testabilité d'un logiciel est le degré auquel un système logiciel ou une unité sous test facilite son propre test.

- En termes simples – à quel point il est difficile de tester et de détecter des défauts dans le logiciel.

- La testabilité est dominée par deux problèmes pratiques :
  - Comment fournir les valeurs de test au logiciel
  - Comment observer les résultats de l'exécution du test

Contrôlabilité et Observabilité

- **Contrôlabilité des logiciels** : Facilité avec laquelle on peut fournir un programme avec les entrées nécessaires, en termes de valeurs, opérations et comportements.
  - Il est facile de contrôler un logiciel avec des entrées provenant de claviers.
  - Les entrées provenant de capteurs matériels ou d'un logiciel distribué sont plus difficiles à gérer

- **Observabilité des logiciels** : Facilité avec laquelle on peut observer le comportement d'un programme en termes de ses sorties, ses effets sur l'environnement et autres composants matériels et logiciels.
  - Les logiciels qui interagissent avec des appareils matériels, des bases de données ou des fichiers distants ont une faible observabilité.

Conditions pour qu'une défaillance soit observée

Trois conditions (RIP) nécessaires pour qu'une défaillance soit observée :

1. **Atteignabilité (Reachability) :** L'emplacement ou les emplacements dans le programme qui contiennent la faute doivent être atteints.

2. **Infection :** L'état du programme doit être incorrect.

3. **Propagation :** L'état **infecté** doit se propager pour provoquer une sortie incorrecte du programme.

Exécution des Tests et Automatisation (1)

- Les tests logiciels sont traditionnellement une **tâche intensive en main-d'œuvre** en raison de la génération manuelle des cas de test, de leur exécution et de l'analyse des résultats.
- **L'automatisation des tests** est le processus d'intégration des valeurs de test dans des scripts exécutables (exécutés automatiquement sur le logiciel testé pour produire un rapport).
- **Meilleure couverture des tests de régression** : L'automatisation permet de créer une bibliothèque de cas de test réutilisables, améliorant ainsi la cohérence et la fiabilité des résultats de test au fil du temps.
- **Facilité de reproductibilité des tests** : Les tests automatisés facilitent la reproduction des résultats en établissant des conditions initiales précises.
- **Journaux de test détaillés** : L'automatisation fournit des journaux détaillés des activités, facilitant le débogage et rendant le processus de test plus organisé et structuré.

Exécution des Tests et Automatisation (2)

- **Efficacité temporelle :** Les tests automatisés peuvent être exécutés sans supervision la nuit, en utilisant les ressources matérielles et de configuration, ce qui permet d'accélérer les tests et de réduire le temps de mise sur le marché.
- **Rentabilité à long terme :** L'automatisation réduit les coûts à long terme, en particulier dans les tests de régression et la maintenance des logiciels pendant la phase de support.
  - Les tâches répétitives, telles que les tests de stress ou de scalabilité, sont plus efficaces et moins coûteuses lorsqu'elles sont automatisées, car elles peuvent fonctionner pendant de longues périodes sans intervention humaine.
- **Prérequis pour l'automatisation des tests :**
  - Cas de test bien définis.
  - Outils et infrastructures disponibles.
  - Professionnels expérimentés en automatisation.
  - Budget adéquat pour l'acquisition des outils.

Limitations de l'Automatisation des Tests

- **Limitations de l'Automatisation des Tests :** L'automatisation ne peut pas remplacer les tests manuels dans les domaines nécessitant la créativité humaine ou l'observation, comme l'ergonomie, l'interopérabilité et certains cas de test au niveau du système.

- **Coûts de Développement et de Maintenance :** Le développement et la maintenance d'une suite de tests automatisés nécessitent des ressources et du temps considérables. Cela doit être géré comme un projet de programmation pour assurer le succès.
  - Un ingénieur en test peut choisir de ne pas automatiser si le coût de l'automatisation dépasse ses avantages, par exemple lorsque le test ne sera utilisé qu'une seule fois ou lorsque les compétences nécessaires en automatisation ne sont pas disponibles.

Équipe de Test : Ingénieur de Test & Gestionnaire de Test

- Ingénieur de Test : Un professionnel IT chargé d'une ou plusieurs activités techniques de test, notamment :
  - Concevoir des entrées de test
  - Exécuter des scripts de test
  - Analyser les résultats
  - Rapporter les résultats aux développeurs et aux gestionnaires

- Gestionnaire de Test : Responsable d'un ou plusieurs ingénieurs de test, il :
  - Définit les politiques et processus de test
  - Interagit avec les autres gestionnaires du projet
  - Aide les ingénieurs à accomplir leur travail

SOFTWARE TESTER

WHO?

Software tester is a specialist who verifies different types of software products by applying particular testing techniques and methods in order to detect bugs, errors, or other issues that can affect user's experience.

WHAT QUALITIES?

- communicative
- imaginative
- analytical
- logical
- attentive
- disciplined
- inquisitive

WHAT TO TEST?

- web
- mobile
- game
- IoT
- VR / AR
- desktop
- cloud-bas
- e-store

WHERE TO WORK?

- Development company
- QA company
- Digital agency
- Freelancing

Objectif d'un testeur logiciel

- … trouver des bugs…
- Aussitôt que possible dans le processus de développement logiciel…
- et s'assurer qu'ils soient corrigés

- **Conseil** : Faites attention à ne pas tomber dans la spirale dangereuse de la perfection inatteignable.

Gestion, maintenance et documentation des tests

- **Gestion des tests** : Définit la politique, organise l'équipe, interagit avec le développement, choisit les critères, décide du niveau d'automatisation nécessaire, …
- Maintenance des tests : Conserver les tests pour réutilisation à mesure que le logiciel évolue
  - Nécessite la coopération des concepteurs de tests et des automatiseurs
  - Décider quand réduire la suite de tests est en partie une question de politique et en partie une question technique – et en général, c'est très difficile !
  - Les tests doivent être intégrés dans des outils de gestion de configuration
- Documentation des tests : Toutes les parties participent
  - Chaque test doit documenter le « pourquoi » – critère et exigence de test satisfaite ou justification pour les tests conçus par des humains.
  - Assurer la traçabilité tout au long du processus.
  - Conserver la documentation dans les tests automatisés

Organisation de l'équipe

- Une organisation de test mature a besoin de concepteurs de tests pour travailler avec les ingénieurs en automatisation des tests, les exécutants et les évaluateurs.
- L'automatisation améliorée réduira le nombre d'exécutants de tests
  - Théoriquement à zéro … mais pas en pratique
- Mettre les mauvaises personnes sur les mauvaises tâches entraîne de l'inefficacité, une faible satisfaction au travail et une faible performance
  - Un concepteur de tests qualifié s'ennuiera avec d'autres tâches et cherchera un emploi en développement
- Les évaluateurs de tests possèdent la connaissance du domaine, ils doivent donc être libres d'ajouter des tests que les processus d'ingénierie "aveugles" ne prendront pas en compte
- Les quatre activités de test sont très différentes

De nombreuses équipes de test utilisent les mêmes personnes pour TOUTES les QUATRE activités !!

Qualités des testeurs de logiciels

Que faut-il rechercher lors d'un entretien pour un poste de testeur de logiciels :

- Sont-ils des explorateurs ?
- Sont-ils des dépanneurs ?
- Sont-ils tenaces (rigides, inflexibles) ?
- Sont-ils créatifs ?
- Sont-ils perfectionnistes (dans la mesure du raisonnable) ?
- Font-ils preuve de bon jugement ?
- Font-ils preuve de tact et de diplomatie ?
- Sont-ils persuasifs (convaincants) ?

Outils de test: Analyse statique

Liste des outils pour l'analyse statique du code :

https://en.wikipedia.org/wiki/List_of_tools_for_static_code_analysis

Exemples:

- **CodeProAnalytiX** (Eclipse based Java software testing tool)
  - https://wiki.eclipse.org/images/7/75/CodeProDatasheet.pdf
- **Checkstyle** (Eclipse plugin)
- **CodeSonar** (commercial tool)
  - Supported languages: C, C++, Java
- **Findbugs** - Find Bugs in Java Programs
- **JavaMetrics (by Semantic Designs)**
  - http://www.semdesigns.com/products/metrics/JavaMetrics.html
  - Calculates metrics for Java programs such as Source Lines of Code (SLOC), number of methods, decision density, and cyclomatic complexity.
- **SonarQube**
- **SpotBugs** (JetBrains):
  - https://plugins.jetbrains.com/plugin/14014-spotbugs

Autres outils de test

Analyze dynamique:

- WinRunner (HP Functional Testing)
  - **Type de test : tests fonctionnels, de charge et de performance**
  - **https://en.wikipedia.org/wiki/HP_WinRunner**
- JMeter
  - **Type de test:** Load and Distributed Testing
  - **Lien vers l' outil:** JMeter

Gestion de tests:

- **Bugzilla**
  - **Type de test :** Defect Tracking
  - **Lien vers l'outil :** Bugzilla
- **Testopia**
  - **Type de test :** Test Case Management
  - **Lien vers l'outil :** Testopia